

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-07-27 | **Scope:** `shende-shweta/FSDKC` (Klearcom Monolithic Platform) — Laravel 12 / PHP 8.3 backend, React 19 + TypeScript + Vite frontend, Node/Express `dev-api`, MariaDB 11 + MongoDB 7, Docker Compose, GitHub Actions CI

Executive Summary

EXECUTIVE SUMMARY

Klearcom is a well-structured polyglot monolith with genuinely strong agentic scaffolding: five `AGENTS.md` guides, a symmetric Discovery/Connect module layout, a working GitHub Actions CI pipeline, and Docker Compose for the full stack. That foundation makes it more agent-ready than most codebases of its age. The debt is concentrated in reproducibility and quality-gate depth rather than architecture. The three most severe gaps are: (1) `backend/composer.lock` is not committed, so every CI run and every developer install resolves PHP dependencies afresh — backend builds are not reproducible; (2) no code-style enforcement exists — no ESLint, Prettier, Laravel Pint, and although `phpstan/phpstan` is declared it has no config file and is never invoked in CI, so agent-authored code has no automated style/lint gate; (3) the database has no migration framework — the entire schema lives in a single `docker/mariadb/init.sql` that only executes on a fresh MariaDB volume, leaving no versioned, reversible path for schema evolution. There are also self-documented debts (a deliberately reintroduced `extract()` pattern, a non-deterministic test using `random_int()`, missing dev-api input validation and rate limiting) captured in `docs/CODEBASE_AUDIT_ISSUES.md`. Overall the repo is **Moderate** readiness: a real CI gate exists and can be trusted for the frontend, but the missing lock file, absent lint layer, and flaky/shallow tests mean agent-authored output cannot yet be verified with full confidence.

Yes TOP-LEVEL .GITIGNORE PRESENT	1 CI/CD WORKFLOWS FOUND	19 / 18 THIRD-PARTY PACKAGES DECLARED / WIRED	Partial .ENV.EXAMPLE PRESENT (BACKEND + DEV-API; FRONTEND MISSING)
--	---	---	--

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

Driven by a missing composer.lock (non-reproducible backend builds), no enforced lint/style gate, no DB migration framework, and shallow/non-deterministic test coverage — no single dimension is High Risk, but every dimension carries real gaps.

Readiness Benchmark Ratings

One row per readiness dimension. "Measured" is the real state found; "Rating" is the band it falls into (worst-wins). This table is the source for the Overall Codebase Rating banner above.

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	R
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	<code>.gitignore</code> + CI + JS lock files present; <code>composer.lock</code> missing, no CODEOWNERS/PR template, CI runs no lint step	
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	18/19 packages wired; <code>phpstan</code> declared but no config and never run	
D3	AI Tool / Agentic Readiness	ready	partial	not ready	5 <code>AGENTS.md</code> + <code>.kiro/</code> config + symmetric enumerable modules; but CI verification gate is shallow (no lint, one flaky test, dev-api untested)	
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	FKs with <code>ON DELETE CASCADE</code> + Mongo indexes present; no migration framework , few secondary indexes, schema+seed combined & non-idempotent	
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose + Dockerfiles + backend/dev-api <code>.env.example</code> ; no lint/formatter enforced , no <code>frontend/.env.example</code>	
D6	Automated Quality Gates & Test Determinism (additional)	lint+type+deterministic tests gate every PR	partial gate	no gate / flaky	CI runs <code>phpunit</code> + <code>tsc --noEmit</code> + <code>vite build</code> ; but no lint, no coverage, <code>ReachabilityCalculationTest</code> uses <code>random_int()</code> (flaky), dev-api has zero tests	

Additional readiness gaps: one additional dimension (D6 — Automated Quality Gates & Test Determinism) was observed beyond the five standard dimensions, driven by the flaky `random_int()` test and the absence of any lint step in CI. No further readiness gaps beyond these were observed.

8.1 Code Repository

Check	Finding	Evidence	Consequence & Next Step
<code>.gitignore</code> coverage	Good. Covers <code>backend/vendor/</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>.env</code> , <code>dev-api/.env</code> , <code>backend/.env</code> , <code>*.log</code> , <code>backend/storage/logs/*.log</code>	<code>.gitignore:1-8</code>	No obvious secret/build-output leakage risk. No change needed.
CI/CD presence	Present but shallow. One workflow: a <code>backend</code> job (composer install → key:generate → phpunit against a MariaDB service) and a <code>frontend</code> job (<code>npm ci</code> →	<code>.github/workflows/ci.yml:1-58</code>	CI verifies backend unit tests and frontend build, but runs no lint/static-analysis step and

	<code>npm run build</code>) on push to <code>main / master</code> and all PRs		never exercises <code>dev-api/</code> (Express service has no CI job). Add a lint stage and a <code>dev-api</code> test job.
Branch protection signals	Missing. No <code>CODEOWNERS</code> (root or <code>.github/</code>), no <code>.github/PULL_REQUEST_TEMPLATE.md</code> , no visible required-checks config	404 on <code>CODEOWNERS</code> , <code>.github/CODEOWNERS</code> , <code>.github/PULL_REQUEST_TEMPLATE.md</code>	Nothing forces review ownership or documents PR expectations; unreviewed changes can land on <code>main</code> . Add a <code>CODEOWNERS</code> mapping <code>backend/</code> , <code>frontend/</code> , <code>dev-api/</code> to owners and a PR template, then require the CI checks on <code>main</code> .
Lock files committed	Partial. <code>package-lock.json</code> present at root, <code>dev-api/</code> , and <code>frontend/</code> ; <code>backend/composer.lock</code> is absent	<code>package-lock.json</code> , <code>dev-api/package-lock.json</code> , <code>frontend/package-lock.json</code> present; 404 on <code>backend/composer.lock</code>	CI's <code>composer install --no-interaction --prefer-dist (ci.yml:38)</code> resolves fresh each run, so a transitive Laravel/MongoDB release can silently change what builds — backend installs are non-reproducible. Run <code>composer update</code> once and commit <code>composer.lock</code> .

8.2 Third-Party Tools Usage

Package	Declared	Actually Wired?	Debt Note
<code>laravel/framework ^12.0</code>	Y	Y	Core framework; routes, controllers, config all use it. Current.
<code>mongodb/mongodb ^2.0</code>	Y	Y	Wired via <code>backend/app/Services/MongoService.php</code> and <code>config/database.php</code> <code>mongodb.uri</code> . Current.
<code>phpunit/phpunit ^11.0</code> (dev)	Y	Y	Invoked in CI (<code>ci.yml:41</code>); 2 unit tests exist. Wired.
<code>phpstan/phpstan ^2.0</code> (dev)	Y	N	Declared as a dev dependency but no <code>phpstan.neon</code> config exists and it is never run in CI — a dead quality gate. Either wire <code>phpstan analyse</code> into CI with a config, or drop the dependency.

<code>express ^4.21</code> (dev-api)	Y	Y	Serves the entire dev API in <code>dev-api/src/server.js</code> . Wired.
<code>cors ^2.8</code> (dev-api)	Y	Y (insecurely)	Wired but with default open config (<code>server.js:17</code> allows all origins) — see <code>CODEBASE_AUDIT_ISSUES.md §9</code> .
<code>mongodb ^6.12</code> (dev-api)	Y	Y	Wired via <code>dev-api/src/mongo.js</code> . Current.
<code>mongodb-memory-server ^10.1</code> (dev-api)	Y	Y	In-memory fallback when no <code>MONGODB_URI</code> (README). Wired.
<code>dotenv ^16.4</code> (dev-api)	Y	Y (partly redundant)	<code>seed.js</code> imports <code>dotenv/config</code> , but <code>package.json</code> dev/seed scripts already pass <code>node --env-file=.env</code> , so <code>dotenv</code> is only half-needed. Minor.
<code>@tanstack/react-query ^5.62</code> , <code>zustand ^5.0</code> , <code>react-router-dom ^7.1</code> , <code>react ^19.2</code> , <code>react-dom ^19.2</code>	Y	Y	All wired (React Query + Zustand for state per README; router in <code>App.tsx</code>). Current, modern versions.
<code>typescript ^5.7</code> , <code>vite ^6.0</code> , <code>@vitejs/plugin-react ^4.3</code> , <code>@types/react*</code> (dev)	Y	Y	Build toolchain; <code>build</code> runs <code>tsc --noEmit && vite build</code> . Wired.

Summary: 19 declared dependencies across the three manifests, 18 genuinely wired. The single unwired item — `phpstan` with no config and no CI invocation — is the actionable debt here.

8.3 AI Tool Usage & Agentic Readiness

Existing AI tooling is present and moderately mature. The repo ships five `AGENTS.md` files — a root guide (`AGENTS.md`), plus module-scoped guides at `backend/app/Modules/Discovery/AGENTS.md` , `backend/app/Modules/Connect/AGENTS.md` , `frontend/src/modules/Discovery/AGENTS.md` , and `frontend/src/modules/Connect/AGENTS.md` . It also carries a `.kiro/settings/mcp-bundles/` directory (Kiro MCP configuration). No `.cursor/` , `.github/copilot*` , or `CLAUDE.md` were found (all 404), so the AI scaffolding is centered on `AGENTS.md` + Kiro.

The guides are concrete and enforceable-in-principle: the root `AGENTS.md` states conventions like "No `extract()` in PHP," "Functional React components only," and "Schema via manual SQL, not Laravel migrations." Notably, the codebase *violates its own first rule* — `backend/app/Legacy/LegacyDataMapper.php` and `LegacyReportController.php` use `extract()` (documented in `CODEBASE_AUDIT_ISSUES.md §2`). This is the clearest example of a rule an agent could be tasked to enforce mechanically.

Enumerable, isolated units of work — strong. The Discovery and Connect modules are near-perfectly symmetric across backend and frontend (controller, model, page, module `AGENTS.md` each), which makes agent-driven refactors highly targetable: "extract business logic from every `*Controller` into a matching `*Service` ," "add a Form Request to every write endpoint," "add an ErrorBoundary per route." The audit doc even enumerates the exact files and line ranges. This is unusually agent-friendly.

The weak link is verification, not enumeration. An agentic harness needs CI to *reject* bad output; here CI runs `phpunit` + `tsc` + `vite build` but no lint/static analysis, one unit test is non-deterministic (`random_int()` in `ReachabilityCalculationTest`), and `dev-api/` has zero automated tests. An agent could produce plausible

changes that pass the shallow gate. **Verdict: partial readiness** — excellent task enumeration and documentation, but the acceptance gate must be hardened (lint + deterministic tests + dev-api coverage) before agent output can be merged with confidence.

8.4 Database Usage

Check	Finding	Evidence	Consequence & Next Step
Schema design — constraints	Good on FKs, thin on indexes. <code>discovery_nodes.discovery_job_id → discovery_jobs(id) ON DELETE CASCADE</code> and <code>connect_check_results.connect_monitor_id → connect_monitors(id) ON DELETE CASCADE ;</code> <code>users.email</code> UNIQUE. But no secondary indexes on FK columns, <code>status</code> , or <code>last_checked_at</code> lookup fields	<code>docker/mariadb/init.sql:38-41,72-75</code>	Referential integrity is enforced in the DB (good), but common filters (by status, by monitor, tree traversal by <code>parent_id</code>) do full scans as data grows. Add indexes on FK and status columns. MongoDB side is better — <code>init.js</code> creates indexes on <code>transcripts</code> and <code>test_events</code> .
Migration hygiene	High-debt. No migration framework at all — the whole schema is one <code>init.sql</code> executed only by MariaDB's <code>docker-entrypoint-initdb.d</code> on a fresh volume . <code>AGENTS.md</code> states this is intentional ("Schema via manual SQL, not Laravel migrations")	<code>docker/mariadb/init.sql:1</code> , <code>docker-compose.yml</code> <code>mariadb volumes</code> , <code>AGENTS.md</code> Conventions	Any schema change after first boot requires manual, unversioned, irreversible SQL against a live DB — no forward/rollback path, no per-change history. This is the single largest DB modernization risk. Adopt Laravel migrations (or a versioned SQL migration tool) and back-fill the current schema as migration 0001.
Data ownership	Moderate. Tables are prefix-scoped by domain (<code>discovery_*</code> , <code>connect_*</code>) but share one flat <code>klearcom</code> schema alongside a generic <code>users</code> table	<code>docker/mariadb/init.sql</code> (all tables in one DB)	Prefixing gives a soft domain boundary, but a single shared schema means extracting Discovery or Connect into its own service later requires untangling shared DB access. Acceptable for a monolith today; document the domain boundaries.

Seed / sample data hygiene	Moderate. <code>init.sql</code> mixes <code>CREATE TABLE IF NOT EXISTS</code> with unconditional <code>INSERT</code> s (schema + seed in one file); <code>docker/mongodb/init.js</code> uses unconditional <code>insertMany</code> . The dev-api path is better — <code>seed.js</code> supports a <code>--force</code> flag via <code>seedMongoData(force)</code> . No production data; no secrets baked into seeds (only local demo rows)	<code>docker/mariadb/init.sql:79-113</code> , <code>docker/mongodb/init.js:1-60</code> , <code>dev-api/src/seed.js</code>	The container seeds are idempotent <i>only</i> because they run once on empty volumes; re-running them elsewhere would duplicate rows. DB credentials (<code>secret</code> / <code>root</code>) are hardcoded in <code>docker-compose.yml</code> and <code>.env.example</code> — fine for local, but must never carry to shared environments. Separate schema DDL from seed data; gate seeds behind an existence check.
----------------------------	---	--	---

8.5 Development Environment

Check	Finding	Evidence	Consequence & Next Step
<code>.env.example</code>	Partial. <code>backend/.env.example</code> and <code>dev-api/.env.example</code> present and match README setup; no <code>frontend/.env.example</code> despite the frontend consuming <code>VITE_API_URL</code>	<code>backend/.env.example</code> , <code>dev-api/.env.example</code> ; 404 on <code>frontend/.env.example</code> ; <code>VITE_API_URL</code> set only in <code>docker-compose.yml</code>	A contributor running the frontend outside CI has no template for <code>VITE_API_URL</code> and read <code>docker-compose.yml</code> to discover <code>frontend/.env.example</code> with <code>VITE_API_URL=http://localhost:8000</code>
OS portability	Good. Everything is container- or Node/PHP-based; <code>docker/php/entrypoint.sh</code> is POSIX <code>sh</code> ; no vendor-specific local dev tool	<code>docker/php/entrypoint.sh:1</code> , <code>docker-compose.yml</code>	Works on Linux/macOS/Windows via Docker native Node+PHP. No portability blocker.
Containerization	Good and current. <code>docker-compose.yml</code> orchestrates nginx 1.27, PHP 8.3-fpm (custom <code>docker/php/Dockerfile</code> with pdo_mysql + mongodb + composer), MariaDB 11 (healthcheck-gated), MongoDB 7, and the Vite frontend; entrypoint auto-installs vendor and generates <code>.env</code> /key	<code>docker-compose.yml:1-77</code> , <code>docker/php/Dockerfile:1-18</code> , <code>frontend/Dockerfile:1-8</code>	Full-stack reproducible environment exists. <code>frontend/Dockerfile</code> runs <code>npm install</code> (not <code>npm ci</code>) so it ignores the committed switch to <code>npm ci</code> for reproducible frontend images.
Code style enforcement	Missing — no enforcement anywhere. No ESLint or Prettier config in <code>frontend/</code> , no	404 on <code>frontend/.eslintrc.*</code> , <code>backend/phpstan.neon</code> ,	Style/quality drifts immediately and inconsistently and — critically for an agentic harness — with no automated gate to catch stylistic or static

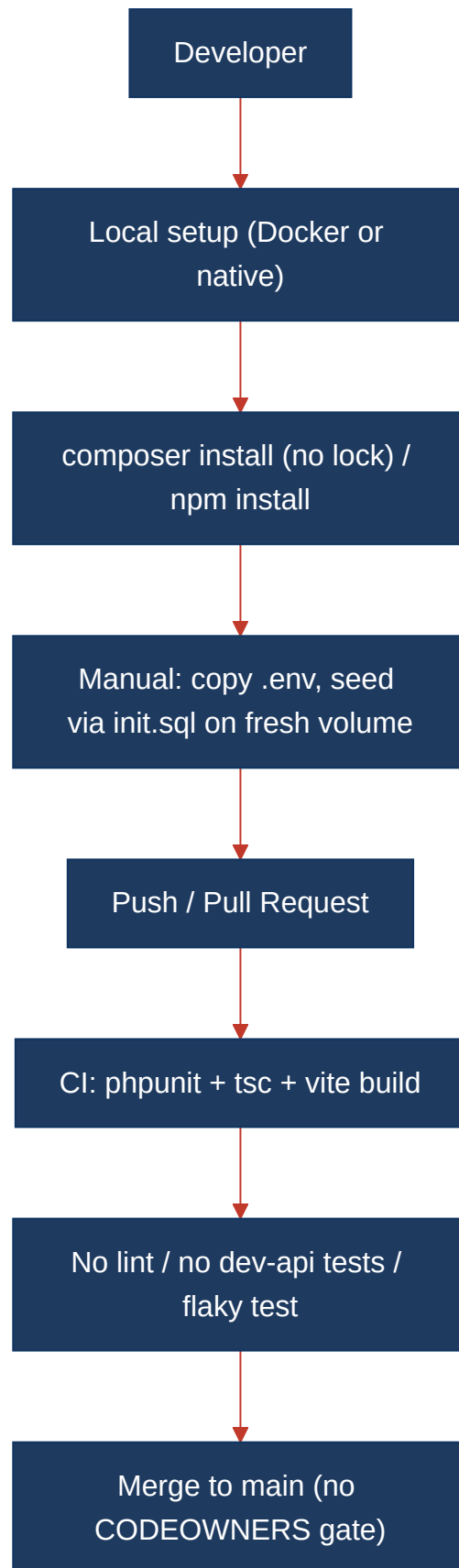
	<code>phpstan.neon</code> or <code>pint.json</code> in <code>backend/</code> ; <code>phpstan</code> is declared but unconfigured and unrun; CI has no lint step	<code>backend/pint.json</code> ; <code>ci.yml</code> has no lint job	analysis regressions in agent-authored code ESLint+Prettier (frontend), Pint + a <code>phpstan.neon</code> (backend), and wire both + a pre-commit hook.
--	---	--	--

8.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Strong — symmetric Discovery/Connect modules, five <code>AGENTS.md</code> , and a fully enumerated debt list in <code>docs/CODEBASE_AUDIT_ISSUES.md</code> with files + line ranges	LOW — already listable; formalize the audit doc into tracked issues/tasks
Isolated, verifiable units of work	Good — module boundaries and file-scoped issues make changes isolatable	MEDIUM — some cross-cutting duplication (<code>buildTree()</code> , reachability formula duplicated 3×) means a single logical fix touches multiple files
CI gate to accept agent-authored output	Partial — CI runs <code>phpunit</code> + <code>tsc</code> + <code>vite build</code> on every PR, but no lint/static analysis, one flaky test, and <code>dev-api</code> untested	HIGH — harden the gate (lint, deterministic tests, dev-api coverage) before trusting agent merges
Repo hygiene for automation (clean checkout, no secrets)	Partial — <code>.gitignore</code> solid and no secrets committed, but <code>composer.lock</code> missing breaks reproducible backend checkout	HIGH — commit <code>composer.lock</code> ; switch <code>frontend/Dockerfile</code> to <code>npm ci</code>
Marketplace packaging readiness	Early — clean module structure and Docker packaging exist, but no versioned release/rollback strategy and no DB migration path	MEDIUM — add migrations + a release/versioning convention before packaging modules for distribution

8.7 Diagrams

Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



8.8 Actions Required

Gap	Action	Rating	Priority
<code>backend/composer.lock</code> not committed — non-reproducible backend builds	Run <code>composer update</code> and commit <code>composer.lock</code> ; keep it in sync going forward	MODERATE	CRITICAL
No code-style / static-analysis enforcement	Add ESLint + Prettier (frontend), Laravel Pint + a <code>phpstan.neon</code> (backend); wire all into CI and a pre-commit hook	MODERATE	HIGH
CI gate too shallow to accept agent output	Add a lint/static-analysis job, a <code>dev-api</code> test job, and make <code>ReachabilityCalculationTest</code> deterministic (remove <code>random_int()</code>)	MODERATE	HIGH
No database migration framework — schema only applies on fresh volume	Adopt Laravel migrations (or a versioned SQL tool); back-fill current schema as migration 0001; add rollback path	MODERATE	HIGH
<code>phpstan</code> declared but never configured or run	Add <code>phpstan.neon</code> and a <code>phpstan analyse</code> CI step, or remove the dependency	MODERATE	MEDIUM
No branch-protection signals (CODEOWNERS, PR template)	Add <code>CODEOWNERS</code> per top-level module and <code>.github/PULL_REQUEST_TEMPLATE.md</code> ; require CI checks on <code>main</code>	MODERATE	MEDIUM
Missing secondary DB indexes on FK / status / lookup columns	Add indexes on <code>discovery_nodes(discovery_job_id, parent_id)</code> , <code>connect_check_results(connect_monitor_id)</code> , and <code>*.status</code>	MODERATE	MEDIUM
Schema DDL and seed data combined; container seeds non-idempotent	Split DDL from seed data; guard seed inserts behind existence checks	MODERATE	LOW
No <code>frontend/.env.example</code> ; <code>frontend/Dockerfile</code> uses <code>npm install</code> not <code>npm ci</code>	Add <code>frontend/.env.example</code> with <code>VITE_API_URL</code> ; switch Dockerfile to <code>npm ci</code>	MODERATE	LOW

8.9 Expected Outcomes

- **Reproducible checkouts** — with `composer.lock` committed and `npm ci` everywhere, every developer, CI runner, and agent works from an identical dependency tree.
- **A trustworthy CI gate** — lint + static analysis + deterministic tests + dev-api coverage mean the pipeline can confidently accept or reject agent-authored changes without human pre-screening.
- **Safe schema evolution** — a versioned migration framework replaces the fragile one-shot `init.sql`, giving forward/rollback paths that unblock database changes in an automated workflow.
- **Enforced review ownership** — CODEOWNERS + required checks ensure agent output lands only through a human review gate on `main`.
- **Foundation for agentic harness & marketplace adoption** — the already-strong module symmetry and `AGENTS.md` scaffolding, once paired with reproducibility and a hardened gate, make Klearcom a realistic early candidate for agent-driven refactoring and module packaging.